

Time-Travelling File System

COL106: Data Structures and Algorithms — Long Assignment 1

Version Control System

Custom Data Structures

C++ Systems Programming

1. What Type of Project Is This?

This is a **Systems Programming + Data Structures project** written in C++. It falls into two overlapping categories that reinforce each other.

Category	Description
Version Control System (Mini Git)	Manages files with full history, branching, snapshots, and rollback — the core concepts behind Git. Everything lives in memory (in-memory VCS).
Data Structures Application	Demonstrates mastery of Tree, HashMap, and Heap by applying them to a real-world problem with practical justification for each choice.
Systems Programming	Written in C++ with manual memory management, pointer-based trees, custom templates, and stdin command parsing.

2. Project Overview

The Time-Travelling File System simulates how a version control system works at its core. Each file is managed as a **tree of versions**. You can edit, snapshot, and roll back to any point in history — just like commits and branches in Git. The system is driven by commands from stdin and runs entirely in memory.

Core idea — how versions work

Every file starts at **version 0** (an automatic snapshot). When you INSERT or UPDATE content:

- If the active version is a **snapshot** (immutable) → a new child version is created (branching).
- If the active version is **not yet snapshotted** → it is edited in-place (no new node).
- SNAPSHOT freezes the active version and stores a message + timestamp.
- ROLLBACK moves the active pointer to any ancestor version (by ID or to the parent).

3. Architecture — File & Layer Breakdown

File / Class	Role
main.cpp	CLI entry point. Reads stdin, parses quoted strings, dispatches commands.
FileSystem (.hpp/.cpp)	Orchestrates all files and implements all 9 commands.
File (.hpp)	One managed file: owns its version tree and its HashMap.
TreeNode (in File.hpp)	One version node: content, timestamps, snapshot flag, parent/child links.
HashMap.hpp	Custom hash map using separate chaining. Template class. O(1) avg lookup.
Heap.hpp	Custom max-heap with comparator. Template class. Powers analytics commands.
compile.sh	One-shot shell script to compile the whole project.
README.md	Instructions, command reference, error-handling decisions for viva.

4. Data Structures — Design & Justification

4.1 Tree (Version History)

Each file's version history is stored as a **tree of `TreeNode` objects**. Every node holds one version of the file content. Nodes have a parent pointer and a vector of children, enabling branching when new versions are created from a snapshot.

- **Why a tree?** Version history is inherently branching — you can roll back to an old snapshot and create a new branch of edits from there.
- DFS traversal over the tree is used by HISTORY to collect all snapshot nodes.
- ROLLBACK simply moves the `active_version` pointer to any node already in the tree — O(1).

Field	Type	Purpose
version_id	int	Unique sequential ID per file, starting at 0.
content	string	The file text at this version.
message	string	Non-empty only for snapshot versions.
created_timestamp	time_t	When this version node was first created.
snapshot_timestamp	time_t	0 if not a snapshot; set when SNAPSHOT is called.
parent	TreeNode*	Pointer to the parent version (null for root).
children	vector	All child versions branching from this node.

4.2 HashMap (Version Lookup)

Each File contains a custom **HashMap<int, TreeNode*>** that maps version IDs to their node pointers. This enables $O(1)$ average-time lookup when the user calls ROLLBACK with a specific version ID — without traversing the whole tree.

- **Implementation:** Separate chaining with a vector of linked-list buckets.
- **Hash function:** $h(\text{key}) = ((\text{key} \% \text{capacity}) + \text{capacity}) \% \text{capacity}$.
- **Collision handling:** Each bucket is a singly linked list of Entry structs.
- On insert, if the key already exists its value is updated (upsert behaviour).
- Templated as **HashMap<K, V>** so it is fully general-purpose.

4.3 Heap (System Analytics)

A custom **Heap<T>** class (comparator-based max-heap) powers the two analytics commands. The same template class is reused with different comparators — no code duplication.

- **RECENT_FILES:** Max-heap ordered by last_modified (most recent file at the top).
- **BIGGEST_TREES:** Max-heap ordered by total_versions (most versions at the top).
- rebuild() heapifies a vector in $O(n)$. sorted_copy() extracts elements in priority order.
- Standard heapify-up (on push) and heapify-down (on pop) give $O(\log n)$ insert/remove.

5. Command Reference

The program reads commands interactively from stdin. Content or messages with spaces must be wrapped in double quotes. Type HELP at the prompt for a quick reference.

Command	Syntax	Data Structure Used
CREATE	CREATE	File registry
READ	READ	File registry
INSERT	INSERT	TreeNode (new child or in-place)
UPDATE	UPDATE	TreeNode (new child or in-place)
SNAPSHOT	SNAPSHOT	TreeNode (marks node immutable)
ROLLBACK	ROLLBACK [versionID]	HashMap (lookup) + TreeNode pointer
HISTORY	HISTORY	TreeNode DFS + sort by timestamp
RECENT_FILES	RECENT_FILES	Heap ordered by last_modified
BIGGEST_TREES	BIGGEST_TREES	Heap ordered by total_versions

6. Key Semantics & Error Handling

The following rules govern how the system behaves. These are the decisions you must be able to explain in the viva.

Situation	Behaviour
CREATE on existing file	Prints error, no change.
INSERT/UPDATE on snapshot version	Creates a new child version (branching). Active pointer advances.
INSERT/UPDATE on non-snapshot version	Edits content in-place. No new node is created.
SNAPSHOT on already-snapshotted version	Prints error, no change.
ROLLBACK without ID	Rolls back to the parent of the active version. Error if at root.
ROLLBACK to non-existent version ID	Prints error, no change. HashMap find() returns nullptr.
READ on empty file	Prints (empty) instead of blank output.
RECENT/BIGGEST with no files	Prints a friendly 'No files in the system.' message.

7. Sample Session

The following shows a typical interactive session demonstrating core features:

```
vfs> CREATE notes.txt
Created file 'notes.txt' (version 0, initial snapshot).

vfs> INSERT notes.txt "Hello, world!"
Inserted into new version v1 of 'notes.txt'.

vfs> SNAPSHOT notes.txt "First draft"
Snapshot taken at version v1 of 'notes.txt': "First draft".

vfs> INSERT notes.txt " More content."
Inserted into new version v2 of 'notes.txt'.

vfs> READ notes.txt
[notes.txt | v2]
Hello, world! More content.

vfs> ROLLBACK notes.txt 1
Rolled back 'notes.txt' to version v1.

vfs> READ notes.txt
[notes.txt | v1 (snapshot)]
Hello, world!

vfs> HISTORY notes.txt
Snapshot history of 'notes.txt':
ID      Timestamp                Message
-----
v0      2025-09-01 10:00:00      Initial version
v1      2025-09-01 10:00:05      First draft
```

8. How to Compile & Run

Using the compile script

```
chmod +x compile.sh
./compile.sh
./vfs
```

Manual compilation

```
g++ -std=c++17 -O2 -Wall main.cpp FileSystem.cpp -o vfs
```

Piping a command file

```
./vfs < commands.txt
```

9. Time Complexity Summary

Operation	Complexity	Reason
CREATE	$O(1)$	Allocates one <code>TreeNode</code> and inserts into registry.
READ	$O(1)$	Reads <code>active_version</code> pointer directly.
INSERT / UPDATE (in-place)	$O(1)$	Mutates the active node's string content.
INSERT / UPDATE (new version)	$O(1)$ avg	new <code>TreeNode</code> + <code>HashMap</code> insert.
SNAPSHOT	$O(1)$	Sets fields on the active node.
ROLLBACK (by ID)	$O(1)$ avg	<code>HashMap</code> lookup by version ID.
ROLLBACK (to parent)	$O(1)$	Follows the parent pointer.
HISTORY	$O(V)$	DFS over V version nodes + sort by timestamp.
RECENT_FILES / BIGGEST_TREES	$O(F \log F)$	Heap rebuild $O(F)$ + extract all $O(F \log F)$.